

GitIntel

Conversational Intelligence for Comprehensive GitHub Repository Analysis

Technical Report

SE - 801 - Software Project Lab III

Supervised by:

Mridha Md. Nafis Fuad

Lecturer

Institute of Information Technology

University of Dhaka

Submitted by:

Md Mostafizur Rahaman (1320)

Institute of Information Technology

University of Dhaka

Submitted to:

SPL-3 Program Committee

Submission Date: January 13, 2026



Institute of Information Technology

University of Dhaka

Letter of Transmittal

13th January, 2026

BSSE 4th Year Exam Committee

Institute of Information Technology

University of Dhaka

Subject: Technical Report Submission - **GitIntel** (Conversational Intelligence for Comprehensive GitHub Repository Analysis).

Sir,

I am respectfully submitting the technical report for our Software Project Lab 3, “**GitIntel: Conversational Intelligence for Comprehensive GitHub Repository Analysis.**”

This report presents a detailed discussion of the project, covering its motivation, system architecture, design decisions, implementation process, and evaluation. GitIntel is designed to analyze GitHub repositories and generate research-quality software engineering datasets and insights in an automated and scalable manner.

I have made my best effort to ensure the accuracy and completeness of this report. I would sincerely appreciate any feedback or suggestions that may help improve the quality of this work.

Thank you for your time and consideration.

Sincerely,

Md Mostafizur Rahaman

BSSE-1320

Institute of Information Technology

University of Dhaka

Supervisor's Signature

Acknowledgement

I would like to express my sincere gratitude to the Institute of Information Technology, University of Dhaka, for providing me with the opportunity to work on the project titled “GitIntel: Conversational Intelligence for Comprehensive GitHub Repository Analysis.” I am deeply thankful to my supervisor, Mridha Md. Nafis Fuad, Lecturer, Institute of Information Technology, University of Dhaka, for his continuous guidance, valuable suggestions, and encouragement throughout the development of this project. His support and feedback played a crucial role in shaping the technical direction and overall quality of the work. I would also like to thank my classmates for their cooperation, discussions, and constructive feedback, which contributed significantly to the completion of this project.

Abstract

Software engineering research increasingly depends on large-scale GitHub repository data, yet extracting research-quality datasets from real-world codebases remains a time-consuming, technically demanding, and error-prone process. GitIntel introduces an autonomous AI-powered solution that transforms any GitHub repository into structured, benchmark-ready datasets in minutes, dramatically reducing the barrier to advanced repository analysis.

The system natively supports seven widely adopted research benchmarks like Defects4J, Bugs.jar, PROMISE, CodeXGLUE, CodeSearchNet, ManySSuBs4J, and Sourcerer while computing comprehensive software metrics across multiple categories including lines of code, CK metrics, complexity measures, and Halstead metrics. This broad metric coverage enables researchers to conduct multifaceted analyses without manual data preparation.

GitIntel's core strength lies in its automated dataset generation capabilities. The system efficiently analyzes repository commit histories to extract genuine bug instances complete with buggy and fixed code pairs, enabling researchers to quickly build training datasets for bug detection, program repair, and code quality studies. A novel multi-LLM jury architecture comprising one generator and three independent judges validates outputs through intelligent rate limiting, loop prevention, and consensus-based verification, ensuring dataset reliability even when processing complex repository histories.

Table of Contents:

1. Introduction:	8
1.1 Purpose	8
1.2 Motivation	8
1.3 Scope	9
1.4 Timeline	9
1.5 Purpose of Document	10
2. Project Description:	10
2.1.1 Normal Requirements	10
2.1.2 Expected Requirements	11
2.1.3 Exciting Requirements	11
2.2 Usage Scenario for GitIntel	12
3. ScenarioBasedModeling:	15
3.1 Use Case Diagram	16
3.1.1 GitIntel	16
3.1.2 Modules of GitIntel	17
3.1.2.1 Repository Clone & Model Selection	18
3.1.2.2 Requirement Specification	18
3.1.2.3 Define Metrics Generation	18
3.1.2.4 Benchmark Dataset Generation	18
3.1.2.5 Multi-LLM Jury Evaluation	19
3.1.2.6 Dataset Generation	19
4. Activity Diagram:	19
5. Data Based Modeling:	21
6. Class-Based Modeling:	21
6.1 Analysis Classes	21
7. Architectural Design:	28
7.1 Architectural Context Diagram	28
7.2 Archetypes	29
7.3 Top Level Components	30
7.4 Deployment Diagram	31
8. Preliminary Test Plan:	33
8.1 High-Level Description of Testing Goals	33
8.2 Test Cases	33
Test Case 1: Repository Cloning and Validation	33
Test Case 2: Invalid Repository Handling	34
Test Case 3: Ask Mode - Interactive Dataset Generation	34

Test Case 4: Agent Mode - Autonomous Dataset Generation.....	34
Test Case 5: 64 Predefined Metrics Computation.....	35
Test Case 6: Custom Metric Generation with Multi-LLM Jury.....	35
Test Case 7: All 7 Benchmark Extraction.....	36
Test Case 8: Dataset Generation - CSV and JSON Formats.....	36
Test Case 9: Performance with Large Repository.....	36
Test Case 10: LLM API Failure and Rate Limit Handling.....	37
Test Case 11: Natural Language Query Understanding.....	37
Test Case 12: Validation Against Known Benchmarks.....	38
Test Case 13: Dataset Integrity and Metadata Validation.....	38
Test Case 14: Temporary Storage Cleanup and Caching.....	38
Test Case 15: Operating System Compatibility.....	39
9. Conclusion.....	39
10. References:.....	40

Figures:

Fig-1: Timeline for GitIntel.....	11
Fig-2: Use Case Level 0 (GitIntel).....	18
Fig-3: Use Case Level 1 (GitIntel).....	19
Fig-4: Activity diagram of GitIntel.....	23
Fig-5: CRC Diagram of GitIntel.....	30
Fig-6: Architectural Context Diagram of GitIntel.....	31
Fig-7: Archetypes of GitIntel.....	32
Fig-8: Top level component of GitIntel.....	33
Fig-9: Deployment Diagram of GitIntel.....	34

Tables:

Table-1: CRC Card (User).....	24
Table-2: CRC Card (Repository).....	25
Table-3: CRC Card (AgenticSystem).....	25
Table-4: CRC Card (MetricsCatalog).....	26
Table-5: CRC Card (LLMCodeJurySystem).....	26
Table-6: CRC Card (BaseExtractor).....	27
Table-7: CRC Card (FormulaDefinition).....	27
Table-8: CRC Card (ProcessingPipeline).....	28

Table-9: CRC Card (AgenticCodeTestExecutor).....	28
Table-10: CRC Card (Dataset).....	29
Table-11: CRC Card (GUI).....	29

1. Introduction

GitIntel is an AI-assisted repository intelligence and dataset generation system designed to support empirical software engineering research. With the rapid growth of open-source software, platforms such as GitHub now host millions of repositories containing valuable information about software evolution, defects, and code quality. However, extracting structured, research-ready datasets from these repositories remains a complex and time-consuming task. GitIntel addresses this challenge by automating the end-to-end process of repository analysis, enabling researchers to generate high-quality datasets efficiently and reliably.

By leveraging static analysis, version control mining, and large language models (LLMs), GitIntel facilitates the extraction of bug instances, software metrics, and benchmark-aligned datasets from large-scale repositories. Different projects, programming languages, and research objectives require different analysis strategies. GitIntel adapts to these variations by supporting multiple research benchmarks, customizable metrics, and intelligent validation mechanisms, making it suitable for diverse software engineering research contexts.

1.1 Purpose

The primary purpose of GitIntel is to assist software engineering researchers in creating structured, reproducible, and publication-ready datasets with minimal manual effort. The system aims to eliminate the traditional bottleneck of dataset preparation by automating repository cloning, commit analysis, bug extraction, and metric computation.

Through the analysis of repository histories and source code artifacts, GitIntel generates verified buggy–fixed code pairs, computes a wide range of software quality metrics, and aligns outputs with established research benchmarks. This enables researchers to focus on analysis and experimentation rather than data engineering. Additionally, GitIntel supports custom metric definitions through natural language, allowing researchers to tailor dataset generation to specific research questions without extensive implementation overhead.

1.2 Motivation

The motivation behind GitIntel arises from the increasing reliance on data-driven methods in software engineering research. Modern studies in defect prediction, program repair, code quality assessment, and software evolution require large-scale, high-quality datasets derived from real-world projects. Unfortunately, constructing such datasets is often labor-intensive, error-prone, and difficult to reproduce.

Many researchers develop ad-hoc scripts to mine repositories, leading to duplicated effort, inconsistent dataset structures, and challenges in result replication. These issues are particularly significant for early-stage researchers and institutions with limited resources.

GitIntel seeks to democratize access to research-grade repository analytics by providing an intelligent, automated system for dataset creation. By integrating multi-LLM consensus-based validation, the system enhances the reliability of extracted bug instances and metrics. This approach enables researchers especially from developing regions to conduct rigorous empirical studies using standardized, trustworthy data, thereby accelerating progress in software engineering research.

1.3 Scope

The scope of GitIntel includes:

- Automated analysis of large-scale GitHub repositories, including projects with extensive commit histories
- Native support for multiple established software engineering benchmarks, each with dedicated dataset schemas
- Computation of a comprehensive set of software metrics covering complexity, size, and defect-related indicators
- Extraction of genuine bug instances with corresponding buggy and fixed code versions from version control systems
- Support for custom, researcher-defined metrics using natural language specifications
- Validation of generated datasets through a multi-LLM jury mechanism to ensure correctness and consistency
- Availability of graphical user interfaces to support diverse research workflows

1.4 Timeline

A 4-month timeline has been planned for this project

Task	Phase 1				Phase 2				Phase 3			
	week 1	week 2	week 3	week 4	week 1	week 2	week 3	week 4	week 1	week 2	week 3	week 4
Prerequisite Knowledge												
Requirements Specification												
Data Collection												
Project Design												
Project Development & Testing												
Final Report Writing												

Fig-1: Timeline for GitIntel

1.5 Purpose of Document

The purposes of this document are

- Identify and analyze the requirements
- Design the architecture
- Design the test plan
- Describe methodology
- Reduce the development effort
- Improve understanding

2. Project Description

The development of GitIntel began with an extensive study of software engineering research methodologies, repository mining practices, and benchmark dataset construction. This analysis revealed a major limitation in existing approaches: although open-source repositories provide rich empirical data, converting raw repositories into structured, research-ready datasets is still largely manual, fragmented, and inefficient. GitIntel addresses this gap by offering an AI-driven automated repository intelligence system capable of analyzing large-scale Git repositories and their full commit histories through comprehensive static analysis.

The system supports both dynamically cloned and pre-existing repositories, computes 64 predefined software metrics, and natively integrates seven established research benchmarks to ensure compatibility and reproducibility. Users can generate customized datasets by combining metrics, benchmarks, and bug-related information, operating in either an interactive “Ask” mode for step-by-step control or an autonomous “Agent” mode for minimal intervention. When required metrics are missing, GitIntel employs LLM-based code generation and validates new implementations through a multi-LLM consensus mechanism, with explicit human intervention only when automated validation fails.

By integrating static analysis, natural language understanding, automated metric generation, and consensus-based validation, GitIntel delivers a robust and adaptive platform for empirical software engineering research. It significantly reduces the time and effort needed to produce reliable, reproducible, and customizable datasets from complex software repositories.

2.1.1 Normal Requirements

Normal requirements define the essential, explicitly expected functionalities of GitIntel. These features establish the core utility of the system.

- **Repository Input Handling:** The system shall accept a Git repository either by cloning it from a remote source or by operating on an already cloned local repository.

- **Static Repository Analysis:** The system shall perform static analysis on source code and commit history.
- **Predefined Metrics Calculation:** The system shall compute 64 predefined software engineering metrics.
- **Benchmark Dataset Support:** The system shall support dataset generation for 7 established software engineering benchmarks.
- **Manual Dataset Configuration:** The system shall allow users to explicitly select metrics, benchmarks, and outputs for dataset generation.

2.1.2 Expected Requirements

Expected requirements represent features users naturally assume will exist in a modern research tool.

- **Natural Language Requirement Interpretation:** The system shall interpret dataset requirements expressed in natural language.
- **Interactive Clarification Workflow:** The system shall request user feedback until all dataset requirements are fully specified.
- **Missing Metric Identification:** The system shall identify whether requested metrics are already available in the system.
- **Controlled Execution Modes:** The system shall provide:
 - an Ask Mode, requiring user confirmation at each major step, and
 - a semi-autonomous execution mode with minimal user intervention.
- **Fully Automated Dataset Generation:** The system shall automatically generate complete datasets from repositories when the requested configuration does not already exist.
- **LLM-Based Metric Code Generation:** The system shall automatically generate missing metric implementations using an LLM.

2.1.3 Exciting Requirements

Exciting requirements introduce innovative, automation-driven capabilities that significantly differentiate GitIntel from conventional repository analysis systems.

- **Multi-LLM Jury Validation:** The system shall validate generated metric implementations through a jury system consisting of three independent LLMs.
- **Automated Unit Test Generation and Execution:** Each jury member shall independently generate and execute unit tests to verify implementation correctness.

- **Consensus-Based Approval Mechanism:** The system shall approve metric implementations only if at least two out of three jury members confirm correctness.
- **Iterative Automated Self-Correction:** The system shall automatically refine failed implementations based on jury feedback, with a maximum of five iterations.
- **Human-in-the-Loop Escalation:** The system shall request human intervention only after automated validation fails.
- **Autonomous Agent Mode:** The system shall execute end-to-end analysis autonomously and interact with the user only when critical ambiguities arise.

2.2 Usage Scenario for GitIntel

GitIntel is a desktop-based, AI-assisted repository intelligence and dataset generation tool designed for software engineering researchers and practitioners. The system enables users to analyze large-scale Git repositories, compute software metrics, and generate research-grade datasets through an interactive graphical user interface (GUI). GitIntel is intended to be used as a standalone research tool on a local machine.

The usage scenario of GitIntel can be described through the following stages:

Tool Launch and Workspace Initialization

Users start GitIntel by launching the desktop application. Upon startup, the system initializes a local workspace that serves as the central environment for repository analysis, dataset configuration, and result management. The GUI presents users with clear options to load repositories, configure analysis parameters, and monitor execution progress.

Repository Selection and Validation

Users provide a Git repository for analysis by either:

- Selecting an existing local repository directory, or
- Providing a remote repository URL, which GitIntel automatically clones.

The system validates the repository structure, programming language compatibility, and commit history to ensure that it is suitable for analysis. Once validated, the repository is prepared for further processing.

Requirement Specification and Mode Selection

Users specify their dataset requirements using natural language within the GUI, such as requesting specific metrics, benchmarks, or bug-related datasets. GitIntel employs a Large Language Model (LLM) to interpret these requirements.

At this stage, users select one of two execution modes:

- **Ask Mode**, where GitIntel requests user confirmation before executing each major step, ensuring transparency and control, or
- **Agent Mode**, where GitIntel operates autonomously and only requests user input when ambiguities or blocking conditions arise.

If the initial requirements are incomplete or unclear, GitIntel iteratively asks for clarification until the dataset specification is fully defined.

Static Analysis and Metric Computation

Once the requirements are finalized, GitIntel performs full static analysis on the repository, including source code and commit history. The system computes 64 predefined software engineering metrics and applies the logic of supported research benchmarks.

If the requested dataset configuration does not already exist, GitIntel generates the dataset from scratch through static analysis without relying on precomputed data.

Automated Metric Generation and Validation

When a requested metric or computation logic is not available, GitIntel automatically generates the required implementation using an LLM-based code generation component. To ensure correctness, the system validates the generated code through a multi-LLM jury mechanism.

Each jury member independently generates unit tests and executes them against the generated implementation. If at least two out of three evaluators approve the result, the metric is accepted. If validation fails, feedback is provided to the generator, which refines the implementation iteratively (up to five attempts). If the metric cannot be validated after these iterations, GitIntel explicitly requests human intervention.

Dataset Generation and Export

After all required metrics and benchmarks are validated, GitIntel generates the final dataset in a structured, research-ready format. The dataset can be exported directly from the GUI in standard formats suitable for empirical analysis and machine learning experiments.

Transparency and Execution Feedback

Throughout execution, GitIntel provides detailed logs and progress indicators within the GUI. Users can review which metrics were computed, which were auto-generated, how validation decisions were made, and where human intervention was required. This ensures transparency, trust, and reproducibility.

Example Scenario Walkthrough

User: A software engineering researcher conducting a comparative defect prediction study.

Scenario Description

The researcher intends to build a dataset that combines:

- Standard benchmark-aligned bug instances,
- Predefined software metrics related to code complexity and evolution, and
- A custom, user-defined defect metric expressed as a mathematical formula.

Step-by-Step Interaction

1. **Tool Launch:** The researcher launches the GitIntel desktop application and initializes a new analysis session.
2. **Repository Selection:** The researcher selects a locally cloned Java repository with a large commit history. GitIntel validates the repository structure and confirms compatibility with supported benchmarks.
3. **Requirement Specification (Natural Language Input):** Through the GUI, the researcher provides the following requirement: “Generate a dataset aligned with Defects4J and Bugs.jar. Include churn, cyclomatic complexity, number of methods, number of classes, and lines of code. Also compute a custom metric: Defect Density = $\text{bug_count} / (\text{lines_of_code} / 1000)$.”
4. **Requirement Interpretation and Validation:** GitIntel’s LLM parses the request and identifies three requirement categories:
 - **Benchmark metrics:** Defects4J, Bugs.jar
 - **Predefined metrics:** churn, cyclomatic complexity, number of methods, number of classes, LOC
 - **Custom metric:** Defect Density

The system verifies that the requested benchmarks and predefined metrics are already supported.

5. **Custom Metric Analysis:** GitIntel checks whether the custom formula “Defect Density” exists in its metric catalog. Since it is not available, the system flags it as a missing metric.
6. **Execution Mode Confirmation:** As the researcher selected **Agent Mode**, GitIntel proceeds automatically without further confirmation.

Automated Metric Generation and Validation

7. **Code Generation for Custom Metric:** GitIntel generates the implementation for the Defect Density metric using an LLM-based code generator.
8. **Multi-LLM Jury Evaluation:** Three independent LLM evaluators:
 - Generate separate unit test files to validate edge cases (e.g., zero LOC, null bug count),
 - Execute the tests against the generated implementation.Need at least two evaluator correctness, if failed then go for previous code generator LLM and provide error and then the LLM rewrite the code, thus the iteration work, The iteration max perform for 5times, if again failed go for human interaction.
9. **Metric Approval:** The Defect Density metric is approved and added to the active analysis pipeline.

Dataset Generation

10. **Static Analysis Execution:** GitIntel performs full static analysis of the repository:
 - Extracts buggy and fixed code pairs aligned with Defects4J and Bugs.jar,
 - Computes churn, cyclomatic complexity, number of methods, number of classes, and LOC,
 - Computes Defect Density using the validated custom formula.
11. **Dataset Assembly**

The system provides benchmark data, predefined metrics, and the custom metric in individual dataset.

Result Review and Export

12. **Execution Transparency:** GitIntel displays detailed logs in the GUI, indicating:
 - Which metrics were pre-existing,
 - Which metric was auto-generated,
 - Jury validation results.
13. **Dataset Export:** The researcher exports the final dataset in a research-ready format for training defect prediction models.

3. Scenario Based Modeling

Scenario-based modeling is an approach used in software development and design to create models that depict how a system behaves or reacts in various real-world situations or scenarios. These scenarios are typically based on user interactions, events, or conditions and help developers understand and visualize how the software will function in different contexts. By simulating these scenarios, developers can better identify potential issues, make informed design decisions, and ensure that the software meets user requirements across different usage contexts.

3.1 Use Case Diagram

A use case diagram is a visual representation in software engineering that illustrates how a system interacts with its external entities or actors to achieve specific goals or tasks. It shows the relationships between use cases (representing system functionalities) and actors (representing external users or systems). Use case diagrams are typically created through requirements gathering, where system functionalities and user interactions are identified, and then diagrammed using specialized software modeling tools

3.1.1 GitIntel

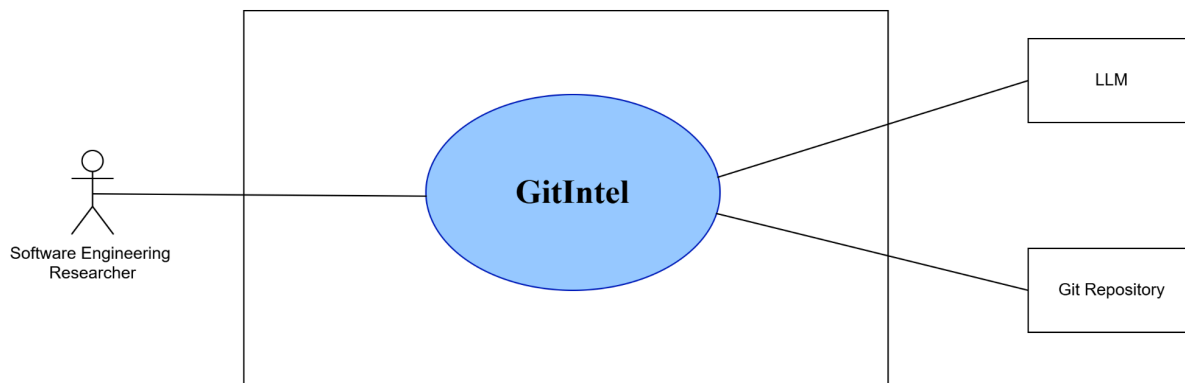


Fig-2: Use Case Level 0 (GitIntel).

Primary actors: Software Engineering Researcher

Secondary actors: Git Repository, LLM

Goal in Context: The diagram shown above represents the Conversational Intelligence for Comprehensive GitHub Repository Analysis (GitIntel)

3.1.2 Modules of GitIntel

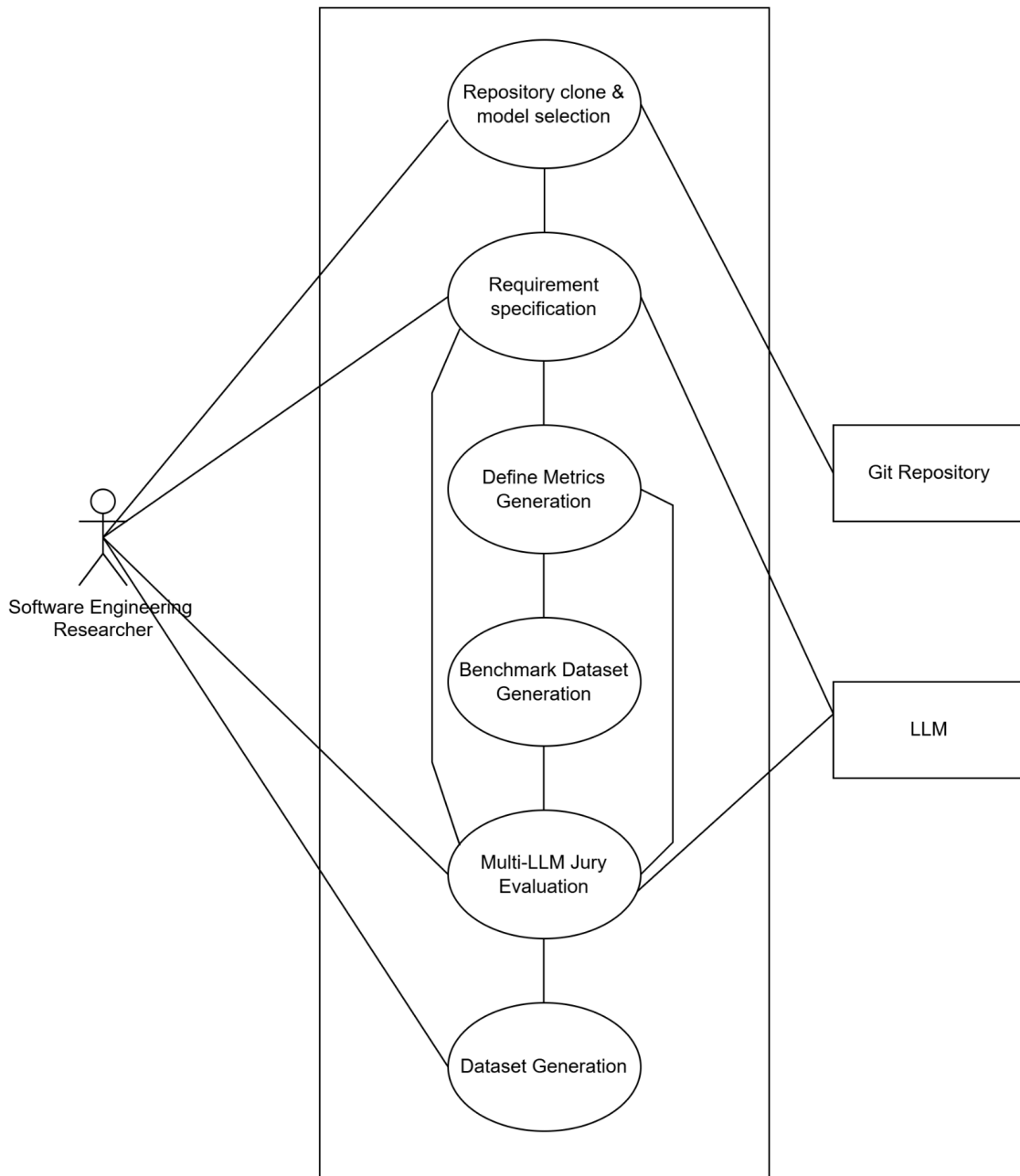


Fig-3: Use Case Level 1 (GitIntel).

Primary actors: Software Engineering Researcher

Secondary actors: Git Repository, LLM

Goal in Context: The diagram shown the modules of the the Conversational Intelligence for Comprehensive GitHub Repository Analysis (GitIntel)

The use case can be elaborated as follows-

3.1.2.1 Repository Clone & Model Selection

The “Repository Clone & Model Selection” module serves as the entry point of GitIntel’s analysis workflow. It allows users to provide a Git repository either by selecting an existing local repository or by supplying a remote repository URL, which GitIntel automatically clones into a controlled local workspace.

This module also supports execution mode selection, allowing users to choose between **Ask Mode**, which requires confirmation at each major step, and **Agent Mode**, which minimizes interaction and proceeds autonomously unless critical ambiguities arise.

3.1.2.2 Requirement Specification

The “Requirement Specification” module enables users to define dataset generation objectives using natural language through the graphical user interface (GUI). Software engineering researchers can specify required metrics, benchmarks, defect-related data, or custom formulas without needing to manually configure low-level analysis parameters.

GitIntel employs an LLM to interpret these requirements and decompose them into structured specifications, such as predefined metrics, benchmark datasets, and missing or custom-defined metrics. If the requirements are incomplete or ambiguous, the system iteratively requests clarification from the user until the specification becomes fully precise.

3.1.2.3 Define Metrics Generation

The “Define Metrics Generation” module is responsible for resolving all metric-related requirements identified during specification. GitIntel first checks whether each requested metric already exists in its internal metric catalog. For predefined metrics, the system directly integrates them into the analysis pipeline.

3.1.2.4 Benchmark Dataset Generation

The “Benchmark Dataset Generation” module aligns repository data with established software engineering benchmarks, such as **Defects4J**, **Bugs.jar**, **ManySStuBs4J**, **CodeXGLUE**, **CodeSearchNet**, **Sourcerer Dataset**, **PROMISE Repository analysis dataset**. GitIntel supports multiple widely used benchmarks, ensuring compatibility with existing empirical studies and reproducibility standards. This module extracts benchmark-specific data, such as buggy and fixed code instances, commit-level annotations, and historical defect information, directly from the repository’s version history. Benchmark logic is applied consistently across repositories, enabling fair comparison and large-scale empirical analysis.

3.1.2.5 Multi-LLM Jury Evaluation

The “Multi-LLM Jury Evaluation” module ensures the correctness and reliability of automatically generated metric implementations. When a new metric is identified then one LLM generates that metrics, three independent LLMs act as evaluators, forming a jury-based validation mechanism. Each evaluator independently generates unit tests targeting edge cases, boundary conditions, and expected behaviors of the metric. These tests are executed against the generated implementation. A metric is approved only if at least two out of three evaluators confirm correctness. If validation fails, structured feedback is provided to the metric generator, which refines the implementation iteratively. This validation-refinement cycle continues for a maximum of five iterations. If the metric still cannot be validated, the system explicitly escalates the issue for human intervention, ensuring transparency and preventing unreliable dataset generation.

3.1.2.6 Dataset Generation

The “Dataset Generation” module assembles all validated metrics, benchmark data, and repository-level features into a structured, research-ready dataset. GitIntel performs full static analysis over the repository’s source code and commits history to compute all required metrics consistently. The final dataset is generated only after all metric implementations predefined or auto-generated have passed validation. Users can export the dataset in standard formats suitable for empirical software engineering research and machine learning experiments. Throughout this process, GitIntel logs execution details, validation outcomes, and metric origins, enabling reproducibility, traceability, and trust in the generated datasets.

4. Activity Diagram

An **Activity Diagram** represents the step-by-step workflow of a system, showing the sequence of activities from start to end along with decision points.

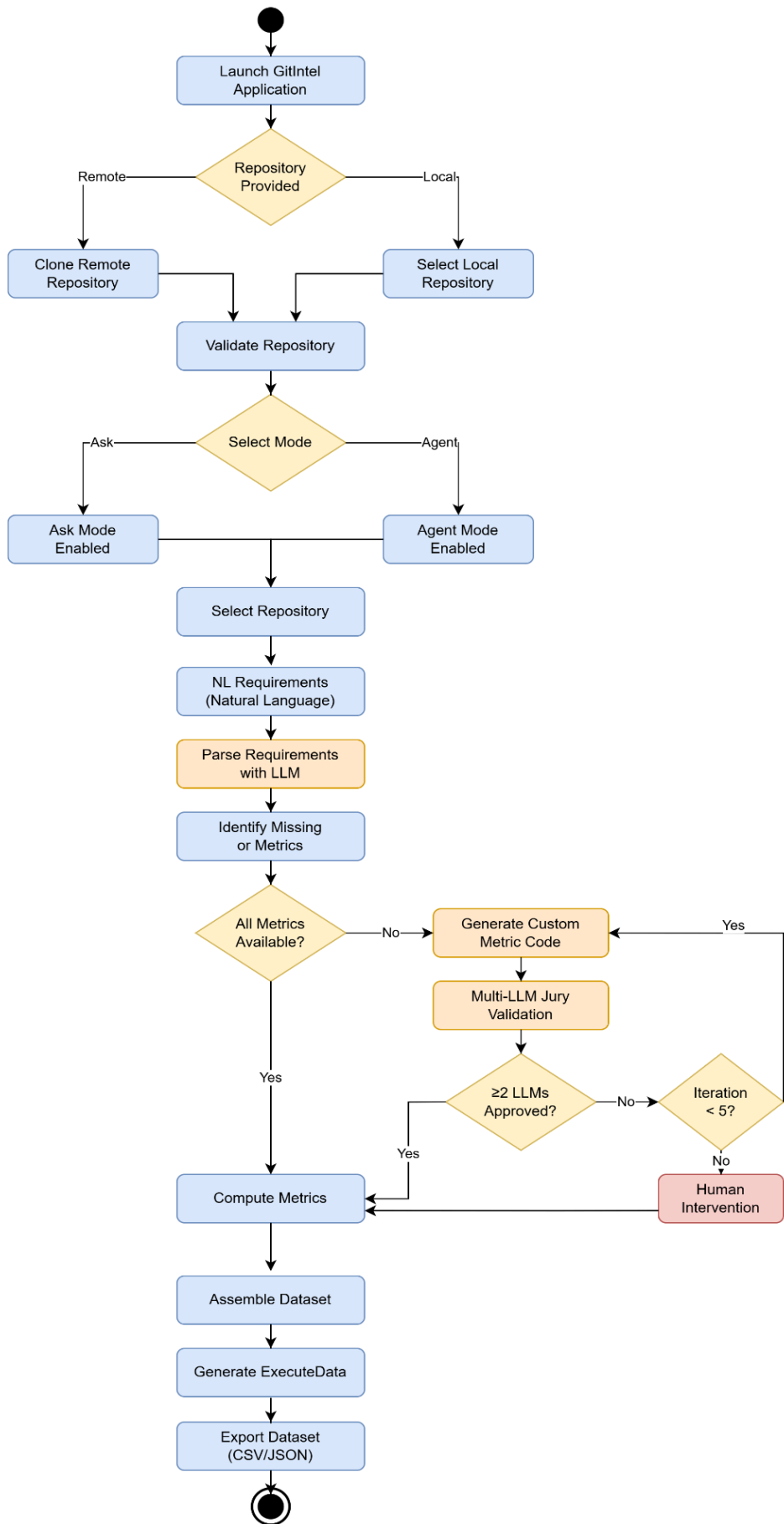


Fig-4: Activity diagram of GitIntel

https://drive.google.com/file/d/1-qA278W8_8PbLGys1Sal39ICVoegVRQi/view?usp=sharing

5. Data Based Modeling

No kind of database or data storage system is used for this tool. So this section does not contain any relevant analysis.

6. Class-Based Modeling

Class-based modeling is a visual representation in software engineering that illustrates the structure of a software system using classes, their attributes, and relationships. It is commonly used in object-oriented design and modeling.

6.1 Analysis Classes

After identifying nouns from the scenario, I filtered nouns belonging to the solution domain using General Classification (External entities, Things, Events, Roles, Organizational units, Places, and Structures). Nouns selected as a potential class were filtered using Selection Criteria (Retained information, Needed services, Multiple attributes, Common attributes, Common operations, and Essential requirements). After performing analysis on potential classes, I have found the following analysis classes:

1. User
2. Repository
3. AgenticSystem
4. MetricsCatalog
5. LLMCodeJurySystem
6. BaseExtractor
7. FormulaDefinition
8. ProcessingPipeline
9. AgenticCodeTestExecutor
10. Dataset
11. GUI

Table-1: CRC Card (User)

User	
Attributes	Methods
- repo_path: str - query: str - preferences: Dict	+ initiate_request(query: str, repo_path: str) + approve_proposal(proposal_id: str) + receive_dataset(dataset: Dataset)

Responsibilities	Collaborators
• Initiate dataset generation requests • Provide repository URLs/paths • Approve/reject proposals • Receive generated datasets	• AgenticSystem • Repository • Dataset

Table-2: CRC Card (Repository)

Repository	
Attributes	Methods
- path: str - commits: List[Commit] - source_code: Dict[str, str] - metadata: Dict	+ get_source_code() -> Dict + get_commits() -> List[Commit] + get_file(file_path: str) -> str + get_file_count() -> int
Responsibilities	Collaborators
• Store source code and commit history • Provide code for analysis • Provide code for metrics calculation • Supply raw data for processing	• BaseExtractor • CKMetricsAnalyzer • UnifiedMetricsAnalyzer • AgenticSystem

Table-3: CRC Card (AgenticSystem)

AgenticSystem	
Attributes	Methods
- mode: AgentMode - repo_path: Path - conversation_history: List[ConversationMessage] - selected_metrics: List[str] - dataset_requirement: DatasetRequirement - sample_generated: bool	+ start_conversation(query: str) -> Dict + continue_conversation(response: str) -> Dict + generate_dataset() -> Dataset + preview_columns() -> ColumnPreview - _extract_repository_data() -> List[Dict] - _request_user_clarification() -> str
Responsibilities	Collaborators

• Coordinate entire dataset generation workflow • Manage user conversation and permissions • Route requests to appropriate analyzers • Validate requirements with LLM jury • Generate column previews for user approval • Execute full dataset generation • Generate final dataset output • Track task progress	• User • UnifiedMetricsAnalyzer • CKMetricsAnalyzer • LLMCodeJurySystem • MetricsCatalog • BaseExtractor • ProcessingPipeline • Dataset
--	--

Table-4: CRC Card (MetricsCatalog)

MetricsCatalog	
Attributes	Methods
- LOC_METRICS: Dict (5) - SIZE_METRICS: Dict (4) - COMPLEXITY_METRICS: Dict (4) - CK_METRICS: Dict (6) - all_metrics: Dict (64)	+ get_metric(metric_name: str) -> Dict + get_category_metrics(category: str) -> Dict + get_all_metrics() -> Dict + validate_metric(metric_name: str) -> bool + get_metric_by_pattern(pattern: str) -> List[str]
Responsibilities	Collaborators
• Maintain catalog of 64 metrics • Organize metrics by 14 categories • Validate requested metrics • Provide metric descriptions and specifications	• AgenticSystem • ProcessingPipeline • Repository

Table-5: CRC Card (LLMCodeJurySystem)

LLMCodeJurySystem	
Attributes	Methods
- generator: GenerativeModel - verifier_1, verifier_2, verifier_3: GenerativeModel - multi_provider: MultiProviderLLM - use_aws_fallback: bool	+ generate_formula_code(description: str) -> str + verify_formula_code(code: str) -> Dict + execute_formula_temp(code: str, test_data: Dict) -> Any + get_verification_consensus() -> bool

- temp_code_dir: Path - attempt_count: int	- _run_unit_tests(code: str) -> TestResult - _fallback_to_aws()
Responsibilities	Collaborators
• Generate code for custom formulas (1 Generator) • Validate generated code (3 Verifiers) • Execute code temporarily and delete • Manage consensus-based approval (2/3 minimum) • Handle rate limiting with AWS Bedrock fallback • Track verification attempts (max 5 iterations)	• AgenticSystem • MetricsCatalog • ProcessingPipeline

Table-6: CRC Card (BaseExtractor)

BaseExtractor	
Attributes	Methods
- source: str - config: Optional[Dict] - extracted_data: List[Dict] - metadata: Dict - temp_dir: Optional[str]	+ extract() -> List[Dict] + validate() -> bool + get_extracted_data() -> List[Dict] + get_record_count() -> int + filter_records(predicate) -> List[Dict]
Responsibilities	Collaborators
• Extract data from 7 benchmark types • Download/clone repositories • Validate source data • Provide extracted records to pipeline • Handle temporary file cleanup	• Repository • AgenticSystem • ProcessingPipeline • MetricsCatalog • Dataset

Table-7: CRC Card (FormulaDefinition)

FormulaDefinition	
Attributes	Methods
- name: str - description: str - formula_code: str - input_schema: Dict - output_type: str - validation_passed: bool - created_at: datetime	No methods (dataclass)
Responsibilities	Collaborators
• Store custom formula definitions • Track formula validation status	• LLMCodeJurySystem • EnhancedAgenticSystem

Table-8: CRC Card (ProcessingPipeline)

ProcessingPipeline	
Attributes	Methods
- processors: List[BaseProcessor] - config: Dict - data: List[Dict] - pipeline_stats: Dict - execution_log: List[Dict]	+ add_processor(processor: BaseProcessor) -> ProcessingPipeline + execute(data: List[Dict]) -> List[Dict] + get_statistics() -> Dict + get_failed_records() -> List[Dict] - _run_processor(processor: BaseProcessor, data: List[Dict])
Responsibilities	Collaborators
• Track processing from console • Handle errors and failed records • Generate processed DataFrame	• BaseProcessor • BaseExtractor • LabelClassifier • TaskManager • Dataset

Table-9: CRC Card (AgenticCodeTestExecutor)

AgenticCodeTestExecutor	
Attributes	Methods
- api_key: str - test_results: List[TestResult] - failing_tests: List[str] - retry_count: int	+ generate_tests(code: str) -> List[str] + execute_tests(code: str, tests: List[str]) -> TestResult + auto_fix_failing_tests(code: str, tests: List[str]) -> str - _parse_test_output(output: str) -> TestResult
Responsibilities	Collaborators
• Generate test cases dynamically for formulas • Execute code against test cases • Report test failures with details • Auto-fix failing code • Persist test results for audit trail	• LLMCodeJurySystem • ProcessingPipeline • Dataset • TaskManager

Table-10: CRC Card (Dataset)

Dataset	
Attributes	Methods
- data: pd.DataFrame - format: str (CSV/JSON) - metadata: Dict - version: str - created_at: datetime	+ export_csv(file_path: str) + export_json(file_path: str) + get_data() -> pd.DataFrame + get_metadata() -> Dict + validate() -> bool
Responsibilities	Collaborators
• Store final generated dataset (CSV/JSON) • Export in multiple formats • Provide data to User • Support dataset metadata and versioning • Enable dataset reuse and caching	• ProcessingPipeline • EnhancedAgenticSystem • TaskManager • Repository

Table-11: CRC Card (GUI)

GUI	
Attributes	Methods
- root: Tk - task_manager: TaskManager - messages: List[AgentMessage] - autonomous_agent: AutonomousAgent - enhanced_system: EnhancedAgenticSystem - repo_path: Optional[str] - selected_metrics: List[str] - execution_complete: bool	+ init(root: Tk) + add_agent_message(type: MessageType, content: str) + update_task_panel(task_status: Dict) + show_column_preview(preview: ColumnPreview) + request_user_approval() -> bool + display_dataset_summary(summary: Dict) - _setup_ui_panels() - _handle_user_input()
Responsibilities	Collaborators
● Display Copilot-style interface ● Show todo list with task progress ● Display agent conversation ● Provide user approval dialog ● Show column preview	● AgenticSystem ● TaskManager

The CRCdiagram of the classes is given below:

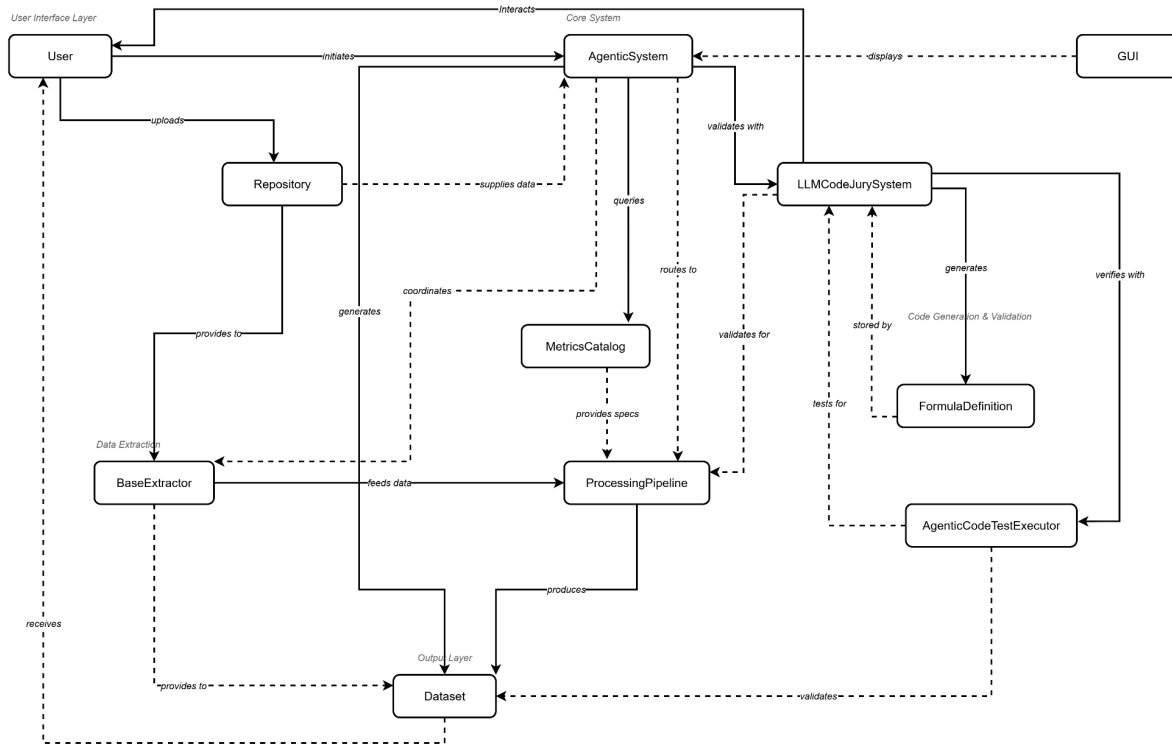


Fig-5: CRC Diagram of GitIntel

<https://drive.google.com/file/d/1YPgEV6axrqct7XjydzN9H-x5YPsO34/view?usp=sharing>

7. Architectural Design

Architectural design is a visual representation in software engineering that outlines the high-level structure and organization of a software system. It focuses on defining the major components or modules of the system, their interactions, and the overall system's architecture.

7.1 Architectural Context Diagram

The architectural context diagram for the tool is given below

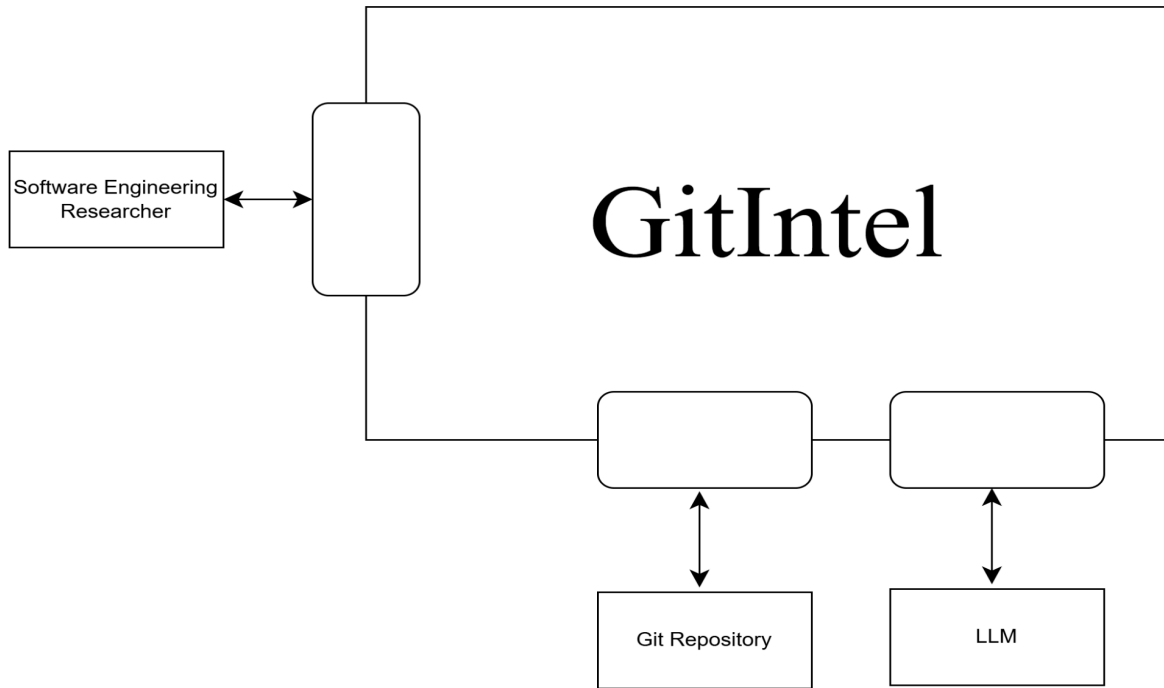


Fig-6: Architectural Context Diagram of GitIntel

In the given diagram:

Primary User - Software Engineering Researcher

Subordinate Systems - Git Repository and LLM Service

The Architectural Context Diagram of GitIntel presents the system as an intelligent research-oriented platform designed to support empirical software engineering analysis. The primary user of the system is the Software Engineering Researcher, while the subordinate systems include the Git Repository and the LLM Service. GitIntel acts as an intermediary between researchers and these underlying systems, enabling automated repository-level analysis and AI-assisted interpretation of software artifacts in a structured and reproducible manner.

Software Engineering Researchers interact with GitIntel to perform large-scale codebase analysis, software metrics extraction, change analysis, and empirical studies using real-world open-source or proprietary Git repositories. GitIntel integrates directly with Git repositories to access source code, commit history, versioning information, and repository metadata for static analysis and dataset generation. Additionally, the integrated LLM Service enhances the system by providing semantic understanding of source code, commit messages, and documentation, supporting intelligent interpretation, natural-language explanations, and advanced insight generation from complex software artifacts.

7.2 Archetypes

The archetypes for the tool are defined below

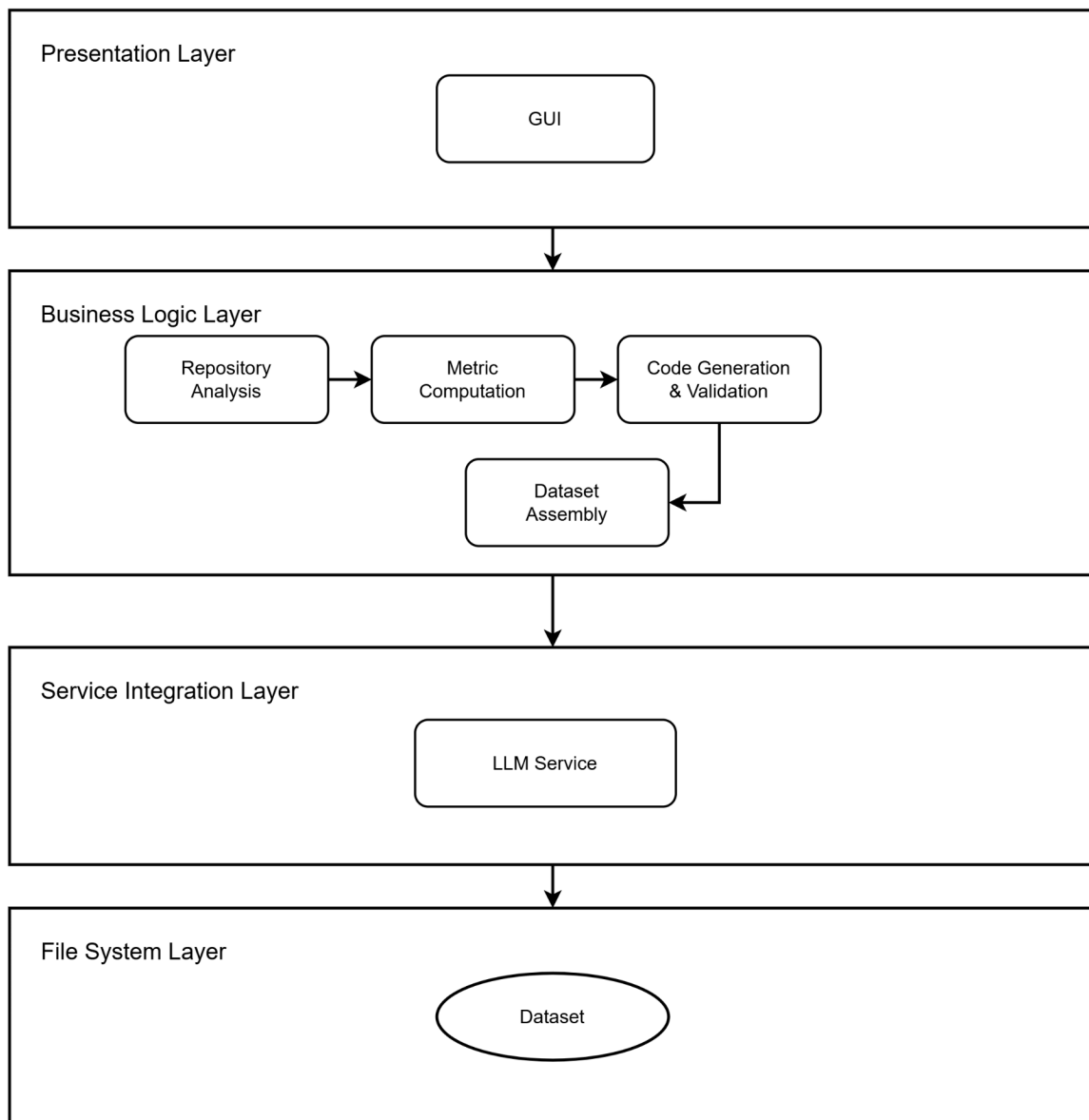


Fig-7: Archetypes of GitIntel

1. File System Layer

- **File Storage:** Central repository for storing all persistent data as files, including cloned repositories, generated datasets (CSV/JSON), temporary code executions, and metadata.

2. Service Integration Layer

- **LLM Service (API Integration):** Manages interactions with Large Language Models (Google Gemini, OpenAI GPT, AWS Bedrock) for natural language processing tasks, code generation, and validation.

3. Business Logic Layer

- **Repository Analysis:** Performs comprehensive static analysis of Git repositories using AST parsing and commit history mining.
- **Metric Computation:** Executes computation of 64 predefined software metrics across 14 categories with dependency-aware calculation.
- **Code Generation & Validation:** Automatically generates Python implementations for custom metrics using LLM-based code generation, validated by multi-LLM jury mechanism.
- **Dataset Assembly:** Integrates analysis results, computed metrics, and benchmark data to construct final research-ready datasets.

4. Presentation Layer

- **User Interface:** Manages all user interactions through Copilot-style GUI with conversational interface and task tracking.

7.3 Top Level Components

Overall architectural structure with top level components is illustrated below

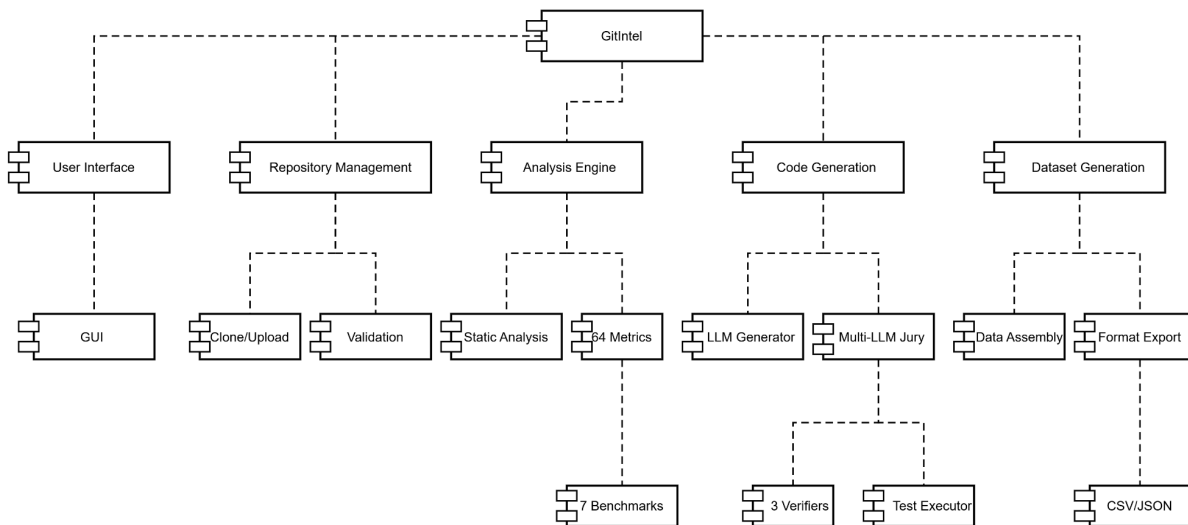


Fig-8: Top level component of GitIntel

<https://drive.google.com/file/d/1DHpyeAb-tYB44caL2DO5pk8eRUHxmPTm/view?usp=sharing>

The GitIntel system is decomposed into five major components:

- **User Interface:** GUI (Copilot) and Authentication
- **Repository Management:** Clone/Upload and Validation
- **Analysis Engine:** Static Analysis, 64 Metrics, and 7 Benchmarks (Defects4J, Bugs.jar, ManySStuBs4J, CodeXGLUE, CodeSearchNet, Sourcerer Dataset, PROMISE Repository)
- **Code Generation:** LLM Generator and Multi-LLM Jury (3 Verifiers, Test Executor)
- **Dataset Generation:** Data Assembly and Format Export (CSV/Json)

7.4 Deployment Diagram

Overall architectural structure with top level components is illustrated below

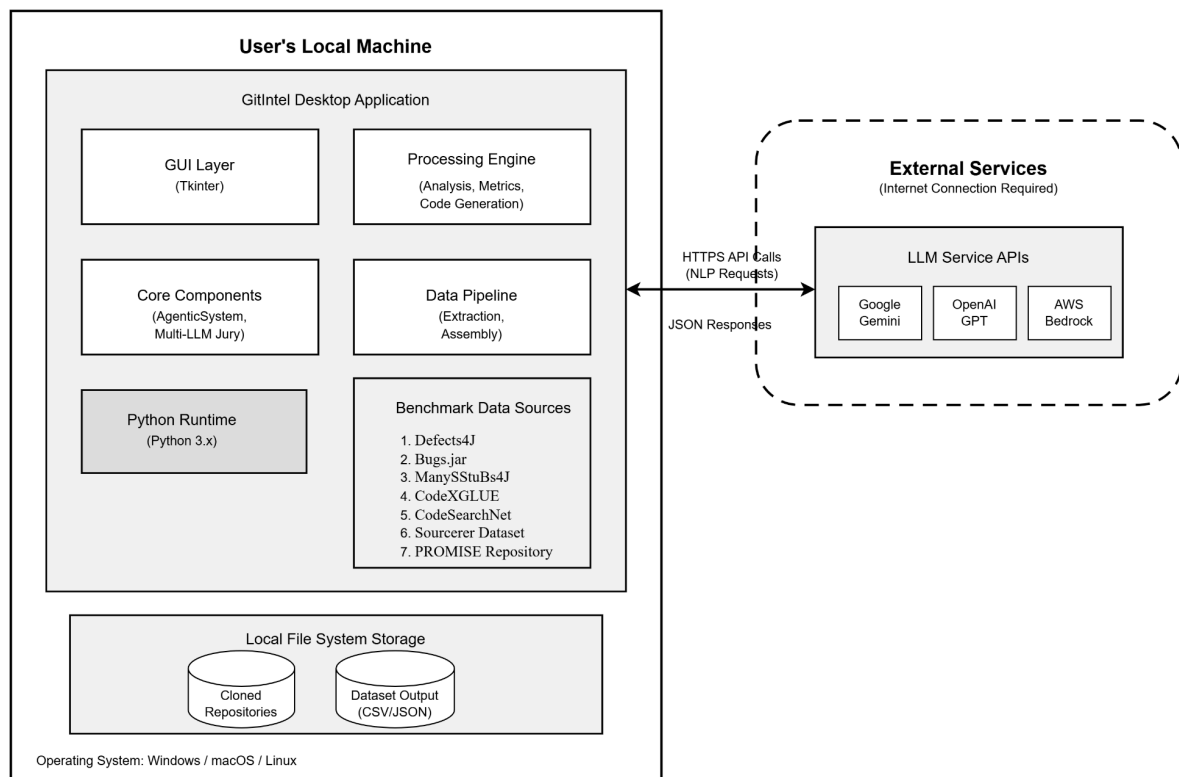


Fig-9: Deployment Diagram of GitIntel

To effectively deploy GitIntel, it's essential to have a clear and straightforward deployment architecture. This ensures scalability, maintainability, and ease of implementation. Above is a deployment diagram showing the desktop application structure with external services.

The deployment architecture consists of:

- **User's Local Machine:** GitIntel desktop application containing GUI Layer (Tkinter/PyQt), Processing Engine, Core Components (AgenticSystem, Multi-LLM Jury), Data Pipeline, and Python Runtime (Python 3.x). Local File System Storage includes Cloned Repositories, Temp Code Storage, and Dataset Output Files (CSV/JSON). Benchmark Data Sources (Defects4J, Bugs.jar, ManySSuBs4J, CodeXGLUE, CodeSearchNet, Sourcerer Dataset, PROMISE Repository) accessed Locally.
- **External Services** (Internet Connection Required): LLM Service APIs (Google Gemini, OpenAI GPT, AWS Bedrock) for NLP and code generation, and **Operating System:** Compatible with Windows, macOS, and Linux platforms.

8. Preliminary Test Plan

This chapter presents a high-level description of the testing goals and a set of test cases designed to evaluate the functionality, performance, and reliability of the GitIntel system.

8.1 High-Level Description of Testing Goals

The primary testing goals for the GitIntel system are as follows:

- To validate that GitIntel meets its functional and non-functional requirements.
- To ensure accurate analysis of Git repositories, including source code, commit history, and repository metadata.
- To verify that software metrics extraction and repository mining are performed correctly.
- To confirm that the LLM service provides meaningful and consistent semantic interpretations of software artifacts.
- To test the system's ability to handle large repositories and multiple analysis requests efficiently.
- To ensure usability and correctness of researcher interactions with the system.
- To validate the consistency and reproducibility of generated analysis results and datasets.

8.2 Test Cases

The following test cases are designed to comprehensively evaluate the GitIntel system.

Test Case 1: Repository Cloning and Validation

Test Scenario: A user provides a Git repository URL for analysis.

Steps:

1. Enter a valid GitHub repository URL (e.g., <https://github.com/elastic/elasticsearch.git>).
2. Initiate repository cloning.
3. Verify repository structure and programming language detection.

Expected Outcome:

- Repository is cloned successfully to local storage.
- Repository metadata is extracted and displayed.
- The user receives a confirmation message.

Test Case 2: Invalid Repository Handling

Test Scenario: A user attempts to clone an invalid or non-existent repository.

Steps:

1. Enter an invalid repository URL (e.g., <https://github.com/invalid/repo123>).
2. Attempt to clone.

Expected Outcome:

- System rejects the invalid URL.
- Error message displays: "Repository not found or URL is invalid."
- The user is prompted to enter a valid repository URL.

Test Case 3: Ask Mode - Interactive Dataset Generation

Test Scenario: User generates a dataset using Ask mode with step-by-step confirmations.

Steps:

1. Select "Ask Mode" operational mode.
2. Provide a repository and specify required metrics (e.g., "LOC, Cyclomatic Complexity, CK metrics").
3. Confirm each major step: Repository analysis, Metric computation, Dataset assembly.
4. Review column preview before final generation.

Expected Outcome:

- The system requests confirmation at each step.
- User approves or modifies selections at each stage.
- Column preview displays all selected metrics.
- Final dataset is generated after user approval.

Test Case 4: Agent Mode - Autonomous Dataset Generation

Test Scenario: User generates a dataset using Agent mode with minimal interaction.

Steps:

1. Select "Agent Mode" operational mode.
2. Provide repository and high-level requirements (e.g., "Generate bug prediction dataset with complexity metrics").
3. System proceeds autonomously.

Expected Outcome:

- System automatically selects appropriate metrics.
- Processing continues without step-by-step confirmations.
- User intervention requested only on ambiguity or errors.
- Dataset is generated automatically and presented to user.

Test Case 5: 64 Predefined Metrics Computation

Test Scenario: System computes all 64 predefined metrics for a Java repository.

Steps:

1. Clone a Java repository (e.g., elasticsearch).
2. Select all 64 metrics for computation.
3. Initiate static analysis.

Expected Outcome:

- All 64 metrics are computed successfully across 14 categories:
 - LOC metrics (5 types), Size metrics (4 types)
 - Complexity metrics (4 types), CK metrics (6 types), etc.
- No computation errors occur.
- Results are displayed with values for each metric.

Test Case 6: Custom Metric Generation with Multi-LLM Jury

Test Scenario: User requests a custom metric not in the predefined 64 metrics.

Steps:

1. Request custom metric: "Calculate ratio of public methods to total methods".
2. LLM Generator creates Python implementation.
3. Three LLM Verifiers independently validate by generating and executing unit tests.
4. System checks consensus ($\geq 2/3$ approval).

Expected Outcome:

- Generator LLM produces Python code implementation.
- Each verifier generates unique unit test cases and executes them.
- If ≥ 2 verifiers approve, metric is integrated.
- If < 2 verifiers approve, code is refined (max 5 iterations).
- User is notified of validation status.

Test Case 7: All 7 Benchmark Extraction

Test Scenario: Test extraction for all supported benchmarks.

Steps:

For each benchmark (Defects4J, Bugs.jar, ManySSuBs4J, CodeXGLUE, CodeSearchNet, Sourcerer Dataset, PROMISE Repository):

- Select a known repository
- Enable respective benchmark extraction
- Verify pattern recognition

Expected Outcome:

- All 7 benchmark patterns are recognized correctly.
- Bug-fix pairs are extracted for each benchmark type.
- Proper labels are assigned (buggy/fixed).
- Dataset includes benchmark-specific metadata.

Test Case 8: Dataset Generation - CSV and JSON Formats

Test Scenario: Generate datasets in both CSV and JSON formats.

Steps:

1. Complete repository analysis with selected metrics.
2. Generate dataset in CSV format.
3. Generate dataset in JSON format.

Expected Outcome:

- **CSV:** File includes proper headers with metric names, all computed values present.
- **JSON:** Proper schema with nested structures, metadata included.
- Both files saved to output directory with download links provided.

Test Case 9: Performance with Large Repository

Test Scenario: Test system performance with a large repository (10,000+ commits).

Steps:

1. Clone a large repository (e.g., Linux kernel or Apache Hadoop or elasticsearch or kafka).
2. Select comprehensive metric set.
3. Initiate analysis and monitor processing time.

Expected Outcome:

- System handles large repository without crashing.
- Progress bar shows incremental updates.
- Analysis completes within acceptable time.
- Memory usage remains within limits.
- Dataset is generated successfully.

Test Case 10: LLM API Failure and Rate Limit Handling

Test Scenario: Test system behavior when LLM API fails or hits rate limit.

Steps:

1. Simulate LLM API unavailability or rate limit.
2. Request custom metric requiring LLM code generation.

Expected Outcome:

- System detects API failure or rate limit error.
- Automatically attempts fallback to AWS Bedrock.
- If all providers fail, error message displays with option to retry.
- User is notified: "Switched to backup LLM provider" (if fallback succeeds).

- System allows proceeding with predefined metrics only if LLM unavailable.

Test Case 11: Natural Language Query Understanding

Test Scenario: Test LLM's ability to understand user requirements expressed in natural language.

Steps:

1. Enter natural language query: "I need a dataset for bug prediction with code complexity and size metrics for Java projects."
2. System uses LLM to interpret requirements.
3. Review system's interpretation.

Expected Outcome:

- LLM correctly identifies:
 - Task: Bug prediction dataset
 - Required metrics: Complexity (4 types) + Size (4 types)
 - Language filter: Java only
- System suggests relevant benchmarks (Defects4J, Bugs.jar).
- User reviews and confirms interpretation.

Test Case 12: Validation Against Known Benchmarks

Test Scenario: Compare GitIntel outputs with established benchmark datasets.

Steps:

1. Analyze a repository from Defects4J with known published metrics.
2. Compare GitIntel's computed metrics with published Defects4J values.

Expected Outcome:

- Metrics match published values within acceptable tolerance (<5% variance).
- Any discrepancies are logged and explained.
- Validates GitIntel's metric computation accuracy.

Test Case 13: Dataset Integrity and Metadata Validation

Test Scenario: Verify dataset integrity and metadata completeness after generation.

Steps:

1. Generate a dataset with 50 metrics.
2. System performs automatic validation checks.
3. Review metadata file.

Expected Outcome:

- Dataset passes integrity checks (no missing values, correct data types).
- Checksums are computed and saved with dataset.
- Metadata includes: repository info, metrics used, benchmarks, timestamp, GitIntel version.
- Metadata is human-readable and machine-parseable.

Test Case 14: Temporary Storage Cleanup and Caching

Test Scenario: Verify temporary file cleanup and caching mechanism.

Steps:

1. Complete a full analysis with custom metric generation.
2. Check temporary storage directories.
3. Re-analyze same repository with overlapping metrics.

Expected Outcome:

- Generated code files are deleted after validation.
- Cloned repositories are removed (if configured).
- System detects previously computed metrics and reuses cached results.
- Overall processing time is significantly reduced on re-analysis.
- User is notified: "Using cached results for X metrics."

Test Case 15: Operating System Compatibility

Test Scenario: Test GitIntel on different operating systems.

Steps:

1. Install and run GitIntel on Windows 10/11, macOS (Intel and Apple Silicon), and Linux (Ubuntu 20.04/22.04).
2. Perform basic analysis on each platform.

Expected Outcome:

- GitIntel runs successfully on all platforms.

- GUI renders correctly on each OS.
- File paths are handled correctly (Windows vs Unix).
- All features work identically across platforms.
- No OS-specific errors occur.

9. Conclusion

In this report, I have documented the necessary technical details required for the development of GitIntel: Conversational Intelligence for Comprehensive GitHub Repository Analysis. The scope, assumptions, QFD, and usage scenarios are thoroughly discussed. Scenario-based modeling, data modeling, and class-based modeling have been done to define the facility and structure of the system. Besides elaborating on the architectural design, it also demonstrates the relationships between the components of the system and external services. It has provided a high-level description about test goals and a summary of features to be tested. Images and diagrams have been provided throughout this document, which make understanding clearer regarding the technical aspects of GitIntel.

While GitIntel is a comprehensive tool for automated repository analysis and dataset generation with AI-powered multi-LLM jury validation, there is still scope for betterment. The future scope could bring the inclusion of expanded metric libraries, additional benchmark support, further advanced AI models to enable more precise code generation, distributed processing capabilities, real-time collaboration features, and integration with research platforms.

The document lays a formal overview of the technical design and working of GitIntel, so that the reader has a complete understanding of the system. It is expected that this report will be very useful for researchers, developers, and stakeholders working in empirical software engineering research and automated dataset generation.

10. References:

- [1] Samuel Abedu, Laurine Menneron, SayedHassan Khatoonabadi, "RepoChat: An LLM-Powered Chatbot for GitHub Repository Question-Answering," 2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR), Ottawa, ON, Canada, 2025
- [2] S. Abedu, A. Abdellatif, and E. Shihab, "LLM-Based Chatbots for Mining Software Repositories: Challenges and Opportunities," EASE '24: Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering, 2024
- [3] Thomas Berkane, Marie-Laure Charpignon, Maimuna Majumder, "LLM-Based Web Data Collection for Research Dataset Creation" medRxiv